# Text Summarization System Report

TF-IDF Extractive and Transformer-Based Abstractive Summarization

## 1. Problem Description

Text summarization is one of the most important and widely studied applications in Natural Language Processing (NLP). In today's information-rich world, the volume of digital text produced daily is enormous — including news articles, research papers, social media posts, and business documents. Reading all of this content manually is impractical, which creates a critical need for automated systems that can condense long texts into shorter, meaningful versions.

The main goal of text summarization is to reduce a long text into a shorter version while preserving the most important information, key facts, and main ideas. A good summary should be concise, coherent, and faithful to the original content — it should not introduce new information or distort the original meaning.

This project implements a complete text summarization system using two fundamentally different approaches to address this challenge:

1. **Extractive Summarization** — Baseline Method
2. **Abstractive Summarization** — Advanced Method

The extractive model works by selecting the most important sentences directly from the original text. It uses statistical scoring (TF-IDF) combined with semantic similarity (sentence embeddings) to rank and select sentences. The abstractive model uses a Transformer-based architecture, specifically the BART (Bidirectional and Auto-Regressive Transformers) model, which reads the entire article and generates entirely new fluent summaries that can paraphrase and synthesize information.

The system was designed to achieve the following objectives:

- Keep important information from the original article
- Reduce the length of the text significantly
- Preserve the meaning and context of the original article
- Compare traditional statistical methods with modern deep learning techniques
- Provide an interactive graphical user interface for easy use

## 2. Dataset Used

The project uses articles and their corresponding human-written summaries (highlights) for both training and evaluation. The system is designed to work with various types of text content, making it versatile for multiple applications:

- **News Articles:** Daily news from major outlets like CNN and DailyMail
- **Blog Posts:** Long-form blog articles and opinion pieces
- **Product Reviews:** Customer reviews and feedback summaries
- **General Long Documents:** Any lengthy text requiring condensation

For this project, we use the **CNN/DailyMail dataset**, which is one of the most widely used benchmarks for text summarization research. The abstractive model specifically uses **facebook/bart-large-cnn**, a version of BART that has been pre-trained and fine-tuned on this exact dataset, giving it strong prior knowledge of news summarization patterns.

**Dataset Statistics:**

- **Source:** News articles from CNN and DailyMail
- **Total Size:** 5000 samples (configurable, full dataset has 300k+)
- **Data Split:** 80% train (4000), 10% validation (500), 10% test (500)
- **Average Article Length:** ~700 words per article
- **Average Highlights Length:** ~50 words per summary
- **Domain:** News journalism (politics, sports, entertainment, world)

Evaluation was performed on 200 test samples to compare the generated summaries from both the extractive and abstractive methods against the human-written reference summaries. This sample size provides stable ROUGE estimates with approximately 95% confidence.

## 3. Preprocessing Steps

Before any feature extraction or summarization can take place, the raw text must be cleaned and transformed into a format suitable for processing. Preprocessing is a critical step that significantly affects the quality of the final summaries. The following preprocessing pipeline is applied to all input text:

**1. Convert text to lowercase**

All characters are converted to lowercase to ensure uniformity. This prevents the system from treating "The" and "the" as different words, which would artificially inflate the vocabulary size.

*Example: "The Food Was Amazing" → "the food was amazing"*

**2. Remove punctuation**

Symbols such as commas, periods, exclamation marks, question marks, and special characters are removed. This allows the system to focus purely on word content without being distracted by punctuation marks that don't carry semantic meaning for scoring purposes.

*Example: "Hello, world!" → "hello world"*

**3. Remove stopwords**

Common function words like "the", "is", "and", "of", "in", and "to" are removed because they appear in almost every document and do not carry important distinguishing meaning. Removing them reduces noise and focuses the model on content-bearing words.

*Example: "the cat is on the mat" → "cat mat"*

**4. Tokenization**

Text is split into individual words (word tokens) and sentences (sentence tokens). Tokenization is the fundamental step that breaks continuous text into discrete units that can be processed, scored, and analyzed by the summarization algorithms.

**5. Sentence Segmentation**

Articles are divided into individual sentences using the NLTK (Natural Language Toolkit) sentence tokenizer. This is especially critical for extractive summarization, where the core task is to rank individual sentences and select the most important ones. The sentence tokenizer handles edge cases like abbreviations ("Dr.", "Mr.") and decimal numbers correctly.

These preprocessing steps collectively improve feature extraction quality and ensure that both the TF-IDF scoring and embedding similarity computations operate on clean, normalized text.

## 4. Feature Extraction

Feature extraction is the process of converting raw text into numerical representations that capture the importance and meaning of sentences. Two complementary feature extraction techniques are used in this system to evaluate sentence importance:

### A) TF-IDF — Term Frequency – Inverse Document Frequency

TF-IDF is a statistical measure that evaluates how important a word is to a document within a collection of documents (corpus). It combines two components:

**Term Frequency (TF):** How often a word appears in a specific sentence. Words that appear more frequently in a sentence are considered more important to that sentence's meaning.

**Inverse Document Frequency (IDF):** How rare a word is across the entire corpus. Words that appear in many documents (like "the", "is") get low scores, while rare, domain-specific words get high scores.

The TF-IDF score for each sentence is calculated by summing the TF-IDF weights of all words in that sentence. Sentences with higher total TF-IDF scores are considered more informative.

**Advantages:**

- Simple and fast computation — no neural networks required
- Effective for extractive summarization tasks
- Easy to interpret and explain — scores are transparent
- Works well on CPU without GPU acceleration

### B) Sentence Embeddings

Sentence embeddings convert entire sentences into dense numerical vectors (arrays of numbers) that capture semantic meaning. Unlike TF-IDF which only counts word frequencies, embeddings understand the actual meaning and context of sentences.

The model used is **all-MiniLM-L6-v2**, a lightweight sentence transformer that produces 384-dimensional vectors. Despite being small (only 80MB), it provides excellent quality for semantic similarity tasks. The model works by encoding each sentence into a vector where semantically similar sentences are close together in the vector space.

Cosine similarity is used to measure how similar each sentence is to the overall document representation (the average of all sentence embeddings). Sentences that are most similar to the document's central theme receive higher scores.

**Advantages:**

- Captures semantic similarity beyond exact word matching
- Understands sentence meaning and context
- Improves sentence ranking quality significantly over TF-IDF alone
- Can recognize that different words can express the same idea

## 5. Baseline Method: Extractive Summarization

The baseline approach uses extractive summarization, which is the simpler and more traditional method. Instead of generating new text, it selects the most important sentences directly from the original article and combines them to form the summary. This approach guarantees that all content in the summary comes from the original text, eliminating any risk of hallucination.

**How it works — Step by Step:**

1. **Sentence Splitting:** The article is split into individual sentences using NLTK's sentence tokenizer.
2. **TF-IDF Scoring:** Each sentence is scored using TF-IDF. The TF-IDF vectorizer is first fitted on training articles to learn the vocabulary, then each sentence's score is computed as the sum of TF-IDF weights of its words.
3. **Embedding Generation:** Each sentence is converted into a 384-dimensional vector using the all-MiniLM-L6-v2 sentence transformer model.
4. **Document Representation:** The document-level embedding is computed as the average (mean) of all sentence embeddings, representing the overall topic.
5. **Cosine Similarity:** The cosine similarity between each sentence embedding and the document embedding is calculated. This measures how well each sentence captures the main theme.
6. **Score Combination:** The TF-IDF score and embedding similarity score are combined using a weighted average formula.
7. **Top-k Selection:** The sentences with the highest combined scores are selected (default k=3, configurable from 1-5).
8. **Order Restoration:** The selected sentences are rearranged in their original order from the article to produce a coherent final summary.

**Scoring Formula:**

$$\textbf{Final Score} = 0.5 \times \textbf{TF-IDF Score} + 0.5 \times \textbf{Embedding Similarity}$$

The weights (0.5 each) can be adjusted through the Streamlit UI to favor either statistical importance (TF-IDF) or semantic relevance (embeddings).

**Advantages:**

- **Fast execution:** ~25 seconds for 200 articles on CPU
- **Preserves original wording:** No modification of source sentences
- **No hallucination risk:** All content is from the original article
- **Easy to debug:** Sentence scores are interpretable
- **Works on CPU:** No GPU required for inference

**Disadvantages:**

- Cannot paraphrase or generate new text
- Sometimes produces disconnected summaries (selected sentences may not flow together)
- Limited by the quality of sentences available in the source
- Cannot combine information from multiple sentences into one

## 6. Advanced Method: Transformer-Based Summarization

The advanced approach uses abstractive summarization with the BART model, which represents the state-of-the-art in neural text generation. Unlike extractive methods that simply select existing sentences, abstractive summarization reads the entire article and generates entirely new text

that captures the key information in a fluent, human-like manner.

**Model Architecture:**

BART (Bidirectional and Auto-Regressive Transformers) is a sequence-to-sequence (seq2seq) model developed by Facebook AI Research. It combines two powerful concepts:

- **Bidirectional Encoder:** Reads the entire input article at once, understanding context from both directions (left-to-right and right-to-left). This is similar to how BERT works.
- **Auto-Regressive Decoder:** Generates the summary one token (word piece) at a time, using the encoded representation and previously generated tokens to predict the next word. This is similar to how GPT works.

The specific model used is **facebook/bart-large-cnn** with 406 million parameters. The "cnn" suffix indicates it has been fine-tuned specifically on the CNN/DailyMail summarization dataset, giving it strong prior knowledge of news summarization style and format.

**Generation Process — Step by Step:**

1. **Tokenization:** The input article is converted into token IDs using the BART tokenizer. Articles longer than 1024 tokens are truncated to fit the model's input limit.
2. **Encoding:** The tokenized article is passed through BART's encoder, which produces contextual representations that capture the meaning of each token in the context of the entire article.
3. **Decoding with Beam Search:** The decoder generates the summary token by token. Instead of greedily picking the most likely next word, beam search maintains multiple candidate summaries (beams=4) and selects the sequence with the highest overall probability.
4. **Length Control:** The generation is constrained by minimum (30) and maximum (130) token limits to ensure summaries are neither too short nor too long.
5. **Repetition Prevention:** No-repeat n-gram constraint (size=4) prevents the model from repeating the same 4-word phrases, improving summary quality.
6. **Decoding:** The generated token IDs are converted back to human-readable text, with special tokens removed.

**Generation Parameters:**

- **Beam Search:** 4 beams (balances quality and speed)
- **Length Penalty:** 1.0 (neutral — neither encourages nor discourages length)
- **No Repeat N-Gram Size:** 4 (prevents repetitive phrases)
- **Early Stopping:** Disabled (allows full beam exploration)
- **Max Length:** 130 tokens (approximately 80-100 words)
- **Min Length:** 30 tokens (approximately 15-20 words)

**Advantages:**

- Generates human-like, fluent, and readable summaries
- Can paraphrase and combine ideas from different parts of the article
- Higher ROUGE scores than extractive (~35% better on ROUGE-1)
- Produces more concise summaries (8.6% compression vs 18.9%)
- Can synthesize information across multiple sentences

**Disadvantages:**

- Much slower than extractive (~505 seconds vs 25 seconds for 200 articles)
- Requires GPU for reasonable inference speed

- May occasionally generate incorrect facts not present in the article (hallucination)
- Less interpretable — harder to understand why specific words were chosen

## 7. Evaluation and Comparison

To objectively measure and compare the quality of summaries produced by both models, we use standard evaluation metrics from the summarization literature. The primary metrics are ROUGE scores, which measure the overlap between generated summaries and human-written reference summaries.

**ROUGE Metrics:**

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is the standard metric for text summarization. It measures n-gram overlap between the generated summary (hypothesis) and the human-written summary (reference). Higher ROUGE scores indicate better quality.

- **ROUGE-1:** Measures unigram (single word) overlap. Indicates whether important individual words from the reference appear in the generated summary.
- **ROUGE-2:** Measures bigram (two-word sequence) overlap. Indicates whether important phrases and word pairs are preserved. This is harder to match than ROUGE-1.
- **ROUGE-L:** Measures the longest common subsequence (LCS). Finds the longest sequence of words that appears in both summaries in the same order (not necessarily contiguous). This captures overall sentence structure quality.

Additionally, we use **BERTScore** as a complementary metric. Unlike ROUGE which only measures exact word overlap, BERTScore uses contextual embeddings from a pre-trained BERT model to measure semantic similarity. This means a paraphrase that uses different words but expresses the same meaning can still receive a high BERTScore, addressing a key limitation of ROUGE.

**ROUGE Evaluation Results:**

| Model | ROUGE-1 | ROUGE-2 | ROUGE-L | Compression Ratio |
|---|---|---|---|---|
| Extractive | 0.2902 | 0.0991 | 0.1878 | 0.1889 |
| Abstractive (BART) | 0.3907 | 0.1692 | 0.2836 | 0.0864 |

**Results Analysis:**

- The abstractive model (BART) achieved significantly better ROUGE scores than the extractive model across all three metrics. ROUGE-1 improved from 0.2902 to 0.3907 (34.6% improvement), ROUGE-2 from 0.0991 to 0.1692 (70.7% improvement), and ROUGE-L from 0.1878 to 0.2836 (51.0% improvement).
- The extractive model was significantly faster, processing 200 articles in approximately 25 seconds compared to 505 seconds for BART (roughly 20x speed difference).
- The abstractive model generated more concise summaries with a compression ratio of 8.6% compared to 18.9% for extractive, meaning BART summaries are about half the length while achieving higher quality scores.

• The extractive model preserved exact wording from the original article, which is advantageous for factual accuracy but limits its ability to produce fluent, readable summaries.

**Additional Comparison Criteria:**

- **Summary Quality:** BART produces more fluent and coherent summaries
- **Compression Ratio:** BART achieves higher compression (shorter summaries)
- **Preservation of Information:** Both models preserve key information, but BART can synthesize across sentences
- **Execution Speed:** Extractive is 20x faster than abstractive

## 8. Manual Evaluation

While automated metrics like ROUGE and BERTScore provide quantitative measures of summary quality, they have known limitations. ROUGE cannot detect grammatical errors, factual inconsistencies, or overall readability. Therefore, manual (human) evaluation was performed to verify summary quality beyond what automated metrics can capture.

**Manual Evaluation Criteria:**

- Does the summary preserve the main idea of the original article?
- Does the summary include the most important information?
- Is the summary significantly shorter than the original text?
- Is the summary readable and fluent (good grammar and flow)?
- Does the summary avoid introducing false information (hallucination)?

**Example Comparison:**

**Original Article Text:**

> *"The food was amazing and the service was excellent. The restaurant was clean and the staff were friendly. However, the delivery was very slow and the order arrived late. Overall, the dining experience was positive despite the delivery issues."*

**Extractive Summary:**

> *"The restaurant was clean and the staff were friendly."*

**Abstractive Summary:**

> *"The food and service were good, but the delivery was slow."*

**Analysis:** The abstractive summary successfully captures all three key points from the original text: food quality ("amazing" → "good"), service quality ("excellent" → "good"), and the delivery problem ("very slow" → "slow"). It does this in a single, fluent sentence. The extractive summary, by contrast, only captures one aspect (restaurant cleanliness and staff) and completely misses the main points about food quality and the delivery issue. This demonstrates the fundamental advantage of abstractive summarization: it can synthesize information from multiple sentences into a concise, comprehensive summary.

This pattern was observed consistently across multiple test examples: the abstractive model tends to produce more informative and readable summaries, while the extractive model sometimes selects sentences that are individually important but collectively incomplete.

## 9. Conclusion

This project successfully implemented a complete text summarization system using both traditional statistical methods and modern deep learning techniques. The system provides two complementary approaches to automatic text summarization, each with distinct strengths and use cases.

The **extractive summarization** approach uses TF-IDF term frequency scoring combined with sentence embedding similarity (all-MiniLM-L6-v2) to identify and select the most important sentences from the original article. This method is fast (~25 seconds for 200 articles), interpretable, and preserves the original wording exactly, making it ideal for applications where factual accuracy is critical and hallucination must be avoided.

The **abstractive summarization** approach uses the BART Transformer model (facebook/bart-large-cnn) to generate entirely new summaries that paraphrase and synthesize information from the source article. This method achieves significantly higher ROUGE scores (34-70% improvement across metrics), produces more fluent and human-like summaries, and generates more concise outputs. However, it is approximately 20x slower and requires GPU acceleration for practical use.

**Key Findings:**

- **Extractive summarization** is best suited for scenarios where speed, interpretability, and factual accuracy are the primary concerns. It is ideal for news monitoring, legal document review, and any application where preserving the exact original phrasing is important.
- **Abstractive summarization** is best suited for scenarios where readability, conciseness, and fluency are prioritized. It is ideal for content creation, email summarization, and applications where a natural, human-like summary is desired.
- The project demonstrates how NLP techniques ranging from classical statistical methods (TF-IDF) to modern Transformer models (BART) can be combined to build powerful, production-ready summarization systems capable of reducing long texts while preserving essential meaning.

Both approaches have their place in the NLP toolkit, and the choice between them depends on the specific requirements of the application — whether speed and accuracy matter more, or whether fluency and conciseness are the priority.

**End of Report**